UNIDAD 9

Trabajo práctico 09 - Actualización y eliminación de reservas de salas

1. Modalidad y entrega

- Individual, por URL accesible del repositorio GitHub.
- Vencimiento: lunes siguiente a las 19:59.
- Punto de partida: copia funcional del TP 08.
- Nombre sugerido: tp-09-salas-crud-mongodb.
- Atlas Free, base reservas_salas_u9 y colección reservas.
- Pruebas de API con Postman de escritorio; vistas y formulario en el navegador.

Conservar la entrega anterior. Copiar fuentes, manifiesto, lockfile y configuración de calidad; preparar dependencias y .env privado en la copia. No entregar secretos ni node_modules.

2. Consigna

Completar el CRUD de reservas: mantener creación y lectura e implementar reemplazo, cambio parcial y eliminación física. Conservar el listado simple y el detalle individual de la unidad anterior. El sitio y la API utilizan la misma base y el mismo servicio asíncrono.

Mantener la estructura de la unidad 8, el arranque después de conectar, la configuración externa y el ajuste DNS autorizado. No insertar automáticamente al iniciar o consultar. Mantener el cierre ordenado del servidor y las herramientas de formato y análisis.

3. Campos y reglas

Campo	Regla conservada
estudiante	Texto obligatorio, trim
email	Texto obligatorio, trim y presencia de @
sala	Sala Norte, Sala Sur o Sala Multimedia
fecha	Texto obligatorio, trim
turno	Mañana, Tarde o Noche
personas	Número entero entre 1 y 6
estado	pendiente o confirmada; default pendiente al crear

El POST conserva los seis campos originales y no acepta que el cliente asigne estado ni _id. PUT exige los siete campos editables, incluido estado. PATCH admite solo personas y estado, con al menos uno presente.

En PUT/PATCH rechazar campos desconocidos, _id, operadores enviados por el cliente, cuerpos ausentes, arreglos y objetos como valores numéricos. Personas se admite como número o texto numérico no vacío, luego se comprueban finitud, rango e integridad. Los controles HTTP y el esquema deben ser compatibles.

No se agregan validaciones de calendario, disponibilidad de sala, solapamiento, cupo compartido o unicidad. Cambiar estado confirma una reserva por decisión explícita; no comprueba esas condiciones.

4. Contrato HTTP

Método y dirección	Resultado
POST /api/reservas	201, recurso y Location; default pendiente
GET /api/reservas/:id	200 y recurso, o 404
PUT /api/reservas/:id	200 y reemplazo completo con el mismo ID
PATCH /api/reservas/:id	200 y cambios parciales permitidos con el mismo ID
DELETE /api/reservas/:id	204 sin cuerpo si eliminó un documento
GET /api/reservas	200 con un arreglo de reservas
ID inválido	400 JSON
ID válido ausente al consultar, modificar o borrar	404 JSON
Entrada o consulta inválida	400 JSON sin modificación
Fallo inesperado	500 JSON seguro

Activar runValidators:true y devolver el documento posterior al modificar. Mantener upsert:false: actualizar un ID ausente no crea. El servicio construye el reemplazo o $set con listas de campos admitidos. No pasar req.body completo al modelo.

Implementar borrado físico de un solo ID. Un segundo DELETE y GET posterior devuelven 404. Documentar la diferencia entre eliminar un documento y cambiar su estado. No se requiere agregar una marca de borrado ni una ruta para vaciar datos.

5. Lectura que se conserva

GET /api/reservas continúa devolviendo un arreglo con las reservas. GET /api/reservas/:id continúa devolviendo una reserva o 404. No cambiar la forma del listado ni agregar metadatos. El HTML conserva su arreglo y /estado sigue mostrando la cantidad real de documentos en MongoDB.

Las consultas avanzadas se trasladan a la unidad 10 y no se implementan ni se evalúan en este TP.

6. Recorrido en Postman

Iniciar el servidor con el entorno de unidad 9. En GET y DELETE dejar Body en none. En POST, PUT y PATCH usar Body, raw, JSON y comprobar Content-Type: application/json.

Crear y guardar identidad

POST a http://localhost:3000/api/reservas con datos ficticios:

```
{
  "estudiante": "Ana Ejemplo",
  "email": "ana@example.com",
  "sala": "Sala Norte",
  "fecha": "2026-10-28",
  "turno": "Mañana",
  "personas": 2
}
```

La fecha es ilustrativa, no una regla de futuro. Registrar 201, ID y Location en Headers de la respuesta. Si Location es relativa, anteponer http://localhost:3000 al consultar. Adaptar el puerto si corresponde.

Reemplazar y modificar

PUT a /api/reservas/ID_RECIBIDO, sustituyendo el marcador por el ID real. Enviar los seis campos anteriores y estado:confirmada. Esperar 200, mismo ID y los siete valores enviados.

PATCH a esa dirección con:

```
{
  "personas": 3
}
```

Esperar 200, personas 3 y los otros campos conservados, incluido estado. Probar también PATCH estado:pendiente. Omitir un campo de PUT debe dar 400; ese rechazo no se completa silenciosamente con un default.

Eliminar con un objetivo comprobado

Elegir una reserva creada para este ejercicio y verificar su ID por GET. Enviar DELETE a la dirección individual. Esperar 204 y cuerpo vacío, comprobar 404 al volver a consultar y verificar que el arreglo del listado tenga una reserva menos. Las otras reservas se conservan.

No borrar datos de TP 08, cambiar accesos de Atlas ni vaciar la colección para fabricar resultados.

7. Persistencia y continuidad

Actualizar una reserva, guardar su ID y valores, esperar el fin de la solicitud y reiniciar con la misma base. Consultar el mismo ID: debe conservar el estado posterior. Eliminar otro documento de ejercicio y reiniciar: debe seguir ausente.

El formulario mantiene creación válida con redirección 302, entrada inválida con 400 y valores conservados. Inicio, CSS, layout, listado, detalle previo y /estado continúan funcionando. Los cambios API deben verse en las consultas HTML existentes.

El servidor mantiene la conexión entre solicitudes. Los programas de terminal cierran sus recursos al terminar. Los errores no exponen trazas ni secretos y las rutas API desconocidas responden JSON.

8. Matriz mínima de pruebas

Caso	Evidencia esperada
POST válido	201, Location y estado pendiente
PUT completo	200, mismo ID, siete campos
PUT incompleto	400 y recurso intacto
PATCH personas o estado	200 y campos omitidos conservados
PATCH vacío, campo extra, _id o $set	400 sin modificación
personas 0, 7, 2.5, true, arreglo, objeto	400 sin modificación
estado o enum inválido	400 sin modificación
ID mal formado	400
ID válido ausente en PUT/PATCH	404 sin creación
JSON mal formado o cuerpo ausente	400 JSON
DELETE existente	204 sin cuerpo; una reserva menos en el listado
Repetir DELETE / GET posterior	404
Reiniciar después de actualizar y eliminar	Cambios persistidos
Sitio, formulario y /estado	Contratos conservados
Fallo inesperado simulado en prueba aislada	500 sin secretos

Registrar resultados observados, el recurso y la cantidad de reservas antes y después. No dar por realizada una prueba pendiente. Para simular un fallo, usar pruebas o un entorno de ejercicio acordado, sin alterar usuarios, acceso o clústeres compartidos.

9. Calidad y documentación

Ejecutar format y check sobre src y scripts. Conservar CommonJS, manifiesto, lockfile, plantilla de entorno y .gitignore.

README: origen de la copia, versiones, entorno de Atlas y DNS, comando de inicio, capas, contratos de PUT/PATCH/DELETE y lectura simple, solicitudes Postman, matriz observada, evidencia de reinicio, criterio de borrado y reglas de validación. Puede incluirse una colección Postman sin secretos como recurso opcional.

No es una entrega de autenticación ni despliegue público. Las consultas avanzadas y las relaciones de datos se planifican en unidad 10.

10. Evaluación

Criterio	Peso
Actualización completa y parcial con validación	40%
Eliminación por ID y contratos de respuesta	25%
Persistencia y continuidad HTML/estado	25%
Calidad, documentación y evidencias	10%

11. Lista antes de entregar

- El docente puede acceder al repositorio y ejecutar la copia.
- El POST sigue creando pendiente y no acepta estado ni _id.
- PUT exige todos los campos y PATCH respeta su lista.
- Los rechazos no modifican ni crean recursos.
- DELETE elimina solo el ID elegido y responde sin cuerpo.
- El listado API y HTML conservan sus arreglos, sin metadatos nuevos.
- GET individual permite comprobar PUT, PATCH y la ausencia posterior a DELETE.
- Los cambios sobreviven al reinicio y /estado refleja MongoDB.
- README, código, lockfile y plantilla están completos.
- No hay secretos, node_modules ni datos personales reales.